

ASSESSMENT 2

- SQL & DATABASE TESTING

SCENARIO 1 :-

You are a junior QA engineer at a food delivery startup. During sprint testing you log a critical defect: the order total is calculated incorrectly when a promo code is applied. The developer fixes the issue and marks it 'Fixed' in JIRA. You pick it up for retesting — but the problem still persists.

Question: What state should the defect move to after your retest reveals the fix was unsuccessful? Walk through the next three steps in the Bug Life Cycle for this defect, naming each state and identifying who is responsible at each step.

Ans :-

After retesting, if the issue still exists, the defect should be moved to the Reopened state.

The next three steps in the Bug Life Cycle are:

Reopened

Who is responsible: QA Engineer (Tester)

What happens: The tester verifies the fix and finds that the defect still occurs. The defect status is changed from Fixed to Reopened, and the issue is reassigned to the developer with details of the failed retest.

Assigned

Who is responsible: Development Lead or Project Manager (or automatically assigned in the tracking tool)

What happens: The reopened defect is assigned back to the appropriate developer for further investigation and correction.

Fixed

Who is responsible: Developer

What happens: The developer identifies the remaining cause of the defect, implements a correct fix, performs unit testing, and updates the defect status to Fixed, making it ready for QA to retest again.

After these steps, the QA engineer retests the fix once more. If the defect is no longer reproducible, its status is changed to Closed. If the issue still persists, the cycle repeats by moving the defect back to Reopened.

SCENARIO 2 :-

You are testing a food delivery app and discover two defects during a sprint: (a) the 'Place Order' button becomes completely unresponsive after a coupon code is applied, blocking all transactions on the platform; and (b) the company logo appears 4 pixels misaligned on the checkout screen.

Question: Assign Severity (S1–S4) and Priority (P1–P3) to both defects and justify each assignment in one sentence. Then explain why a defect can have high Severity but low Priority use defect (a) or (b) to illustrate your answer.

Ans :-

Defect (a): "Place Order" button becomes unresponsive after applying a coupon code

Severity: S1 (Critical)

Justification: This defect blocks users from placing orders, making a core business function unusable.

Priority: P1 (High)

Justification: It directly impacts customers and revenue, so it must be fixed immediately.

Defect (b): Company logo is 4 pixels misaligned on the checkout screen

Severity: S4 (Low/Minor)

Justification: The issue is purely cosmetic and does not affect the application's functionality.

Priority: P3 (Low)

Justification: Since it has minimal impact on users and business operations, it can be fixed in a later release.

Why a defect can have high Severity but low Priority

SCENARIO 3 :-

Your QA lead asks you to automate the regression suite for a food delivery app's search and filter functionality. The feature is updated every sprint, manual regression takes 4 hours per cycle, and the team runs it twice per sprint.

Question: Justify why this scenario is a strong candidate for Selenium IDE automation using at least two criteria from the 'when to automate' framework. Then state two limitations of Selenium IDE that your lead must consider before committing to it as the automation tool.

ANS.

Why is this a good candidate for automation?

1. Repetitive Testing

Regression testing is executed twice every sprint, making it repetitive. Automating these tests saves significant manual effort and reduces execution time.

2. Stable Functionality

Although the feature receives updates, the core search and filter workflow remains largely consistent. Stable, frequently executed tests are ideal candidates for automation.

Limitations of Selenium IDE

1. Limited Programming Capability Selenium IDE does not support advanced programming logic such as complex loops, custom frameworks, or sophisticated data-driven testing.
2. Limited Cross-Browser and Integration Features.

Although Selenium IDE supports major browsers, it is less flexible than Selenium WebDriver for advanced browser configurations, CI/CD integration, and large-scale automation projects.

SCENARIO 4 :-

You are building a Selenium IDE test for the food delivery app's checkout page. The 'Confirm Order' button has both an id attribute ('btn-confirm-order') and an XPath ('//div[@class='checkout-wrapper']/button[3]'). The development team frequently restructures the checkout page DOM, but the id value remains stable across all releases.

Question: Which locator should you use and why? Define what makes a locator 'fragile' in Selenium IDE automation, and describe one specific consequence of using a fragile locator in a regression suite that runs every sprint.

Ans :-

The ID locator should be used.

Locator:-

id=btn-confirm-order

Reason :-

An ID is unique, faster to locate, and remains stable across releases. Since the development team frequently changes the DOM structure, using XPath would make the test more vulnerable to failures.

What makes a locator fragile?

A locator is considered fragile when it depends on elements that change frequently, such as:

- Absolute XPath
- Element position
- Dynamic class names
- Deep DOM hierarchy

Consequence of using a fragile locator :-

If the checkout page layout changes during a sprint, the XPath may no longer locate the button even though the application works correctly. This results in false test failures, increasing maintenance effort and reducing confidence in the regression suite.

SCENARIO 5 :-

You are performing backend data validation for a food delivery app. The order history page displays the customer name, restaurant name, and order amount. These values are stored in three separate tables: users (user_id, name), restaurants (restaurant_id,name), and orders (order_id, user_id, restaurant_id, amount, status).

Question: Describe the JOIN structure needed to validate whether the order history page is displaying accurate data. Explain why a LEFT JOIN may be more appropriate than an INNER JOIN for this validation task, and what discrepancy it would help you detect.

Ans.

JOIN Structure

```
SELECT  
o.order_id,  
u.name AS customer_name,
```

```
r.name AS restaurant_name,  
o.amount,  
o.status  
FROM orders o  
LEFT JOIN users u  
ON o.user_id = u.user_id  
LEFT JOIN restaurants r  
ON o.restaurant_id = r.restaurant_id;
```

Why use LEFT JOIN instead of INNER JOIN?

A LEFT JOIN returns all records from the Orders table, even if matching user or restaurant records are missing.

Discrepancy Detected

Suppose an order exists in the Orders table, but its corresponding restaurant record was accidentally deleted.

INNER JOIN: The order disappears from the result set, hiding the problem.

LEFT JOIN: The order is still displayed, while the restaurant columns become NULL, clearly indicating a data integrity issue.

Therefore, LEFT JOIN is better for backend validation because it helps QA identify missing or inconsistent related data.

SCENARIO 6 :-

You are performing database testing for the payment module of a food delivery app. During a test run, you observe that the wallet balance was debited from the user's account, but no corresponding order record was created in the orders table. You suspect a partial transaction failure.

Question: Identify which ACID property has been violated in this scenario and explain why. Describe two SQL queries you would write to

confirm whether the violation occurred, and explain what this finding reveals about the importance of ACID compliance in QA testing.

Ans.

ACID Property Violated

The violated property is Atomicity.

Reason

Atomicity requires that all operations in a transaction either complete successfully or none of them are applied.

In this case:

Wallet balance was deducted.

Order record was not created. This indicates a partial transaction, violating Atomicity.

SQL Query 1

Verify wallet deduction.

```
SELECT
user_id,
wallet_balance
FROM users
WHERE user_id = 101;
```

This confirms whether the wallet amount was deducted.

SQL Query 2

Verify order creation. SELECT *

```
FROM orders
WHERE user_id = 101
ORDER BY order_id DESC;
```

If no new order record exists while the wallet balance has decreased, the transaction failed partially.

Importance of ACID Compliance

ACID properties ensure database consistency and reliability.

Without Atomicity:

- Customers may lose money.
- Orders may not be recorded.
- Business reports become inaccurate.
- Customer trust decreases.
- Financial reconciliation becomes difficult.